

JSON++: Making Configuration Reuse Declarative

Hiroshi Ukai
LegalOn Technologies
Shibuya, Tokyo

hiroshi.ukai@legalontech.jp



Abstract—Configuration files describe infrastructure, deployment environments, and application behavior. As systems grow, these files often accumulate duplicated fragments across services and environments, making them harder to maintain. Existing solutions such as templates and overlays reduce some duplication but often add another language layer or a tool-specific model.

This article introduces *JSON++*, a lightweight extension of JSON for declarative structural reuse. JSON++ adds two minimal abstractions: inheritance for reusing configuration structures and expression evaluation for computing derived values. We present *jq++*, a tool that evaluates JSON++ configurations and produces ordinary JSON. Because *jq++* accepts JSON-compatible input and emits standard JSON, it fits into existing configuration pipelines without changing upstream generators or downstream consumers.

Our experience suggests that small, compatible extensions to widely used data formats can improve configuration reuse substantially while avoiding the cost of adopting a new configuration language.

1 INTRODUCTION — THE PROBLEM OF CONFIGURATION DUPLICATION

Modern software systems rely heavily on configuration files to describe infrastructure, deployment environments, and application behavior. These artifacts appear in forms such as Kubernetes manifests, CI/CD pipelines, infrastructure definitions, and application runtime settings. They are commonly written in formats such as JSON [1], YAML [2], and HCL [3].

Although these formats represent structured data effectively, they provide limited support for structural reuse. As systems grow, engineers often duplicate configuration fragments across services, environments, and repositories. Even small changes must then be propagated manually, increasing maintenance effort and the risk of inconsistency.

The problem is especially visible in infrastructure configuration. For example, Kubernetes deployments often share runtime settings across multiple workloads while also varying across development, staging, and production. Existing mechanisms such as overlays, templates, and application-specific configuration languages address parts of this problem, but they often force developers to reason about generation logic in addition to the resulting configuration.

Several tools address duplication by introducing dedicated configuration languages. Examples include Jsonnet [4]

and CUE [5]. These approaches are expressive, but they require developers to adopt languages and toolchains distinct from JSON. Even when configuration data is conceptually JSON-like, developers must learn additional syntax and semantics.

This article explores a different approach. Instead of introducing a new configuration language, we extend JSON itself with a small set of declarative reuse mechanisms. The resulting notation, called *JSON++*, preserves compatibility with JSON while enabling configurations to inherit structures and compute values dynamically.

JSON++ adds two minimal abstractions. Configurations may extend other configurations through an inheritance-like mechanism with selective overrides, and values may be defined through evaluable expressions. Together these mechanisms allow complex configuration relationships to be expressed concisely while remaining within the JSON ecosystem.

To demonstrate the practicality of this approach, we present *jq++*, a tool that evaluates JSON++ configurations and produces ordinary JSON. Because both the input and output remain JSON-compatible, *jq++* can be integrated into existing pipelines as a preprocessing step. Editors, validators, and deployment systems continue to operate on standard JSON without modification.

Contributions

This article makes three contributions.

- It identifies a practical limitation of widely used configuration formats: the lack of direct support for structural reuse within configuration data.
- It introduces JSON++, a lightweight extension of JSON that provides declarative inheritance and expression evaluation while preserving compatibility with the JSON data model.
- It presents *jq++*, a tool that implements the JSON++ evaluation model and integrates into existing configuration pipelines by emitting ordinary JSON.

The remainder of the article is organized as follows. Section 2 presents a motivating example. Section 3 describes the design principles behind JSON++. Section 4 introduces

the jq++ evaluation pipeline. Section 5 discusses implementation experience. Section 6 shows how JSON++ fits existing ecosystems. Sections 7 and 8 summarize observations and lessons learned. Section 9 discusses related work, and Section 10 concludes.

2 A MOTIVATING EXAMPLE FROM KUBERNETES CONFIGURATION

Configuration duplication becomes visible quickly in systems deployed across multiple environments. Consider a web application deployed to Kubernetes in development, staging, and production. The system consists of five workloads: a backend service, an admin service, a scheduled job service, a frontend, and a database.

A common organization is to maintain a base directory together with overlays for each environment, a structure widely used in Kubernetes [6], [7] repositories and tools such as Kustomize [8].

As Figure 1 shows, several workloads often share identical runtime fragments. Listing 2 shows a typical base-and-overlays structure used with Kustomize.¹

Listing 1. Typical base-and-overlays structure used with Kustomize

```
base/
  backend.yaml
  admin.yaml
  job.yaml
  frontend.yaml
  db.yaml

overlays/
  dev/
    kustomization.yaml
  stg/
    kustomization.yaml
  prd/
    kustomization.yaml
```

This structure captures differences across environments. For example, a production overlay may increase replica counts or storage sizes. However, many fragments are shared not only across environments but also across related workloads within the same system.

For instance, the backend, admin, and scheduled job services often share database connectivity and cache endpoints. The backend and admin services may also share HTTP health probes. As a result, similar fragments appear repeatedly across manifests.

Listing 2. Repeated runtime configuration across multiple workloads

```
# backend.yaml
containers:
- name: backend
  image: example/backend:1.0
  env:
  - name: DB_HOST
    value: postgres
  - name: DB_PORT
    value: "5432"
  - name: REDIS_HOST
    value: redis
  - name: LOG_LEVEL
```

1. Examples are shown in YAML for readability. YAML documents are typically convertible to JSON data structures, and many systems internally operate on JSON-like data models. Because JSON++ operates on the JSON data model, these examples can be interpreted equivalently as JSON.

```
    value: info
  livenessProbe:
    httpGet:
      path: /health
      port: 8080

# admin.yaml
containers:
- name: admin
  image: example/admin:1.0
  env:
  - name: DB_HOST
    value: postgres
  - name: DB_PORT
    value: "5432"
  - name: REDIS_HOST
    value: redis
  - name: LOG_LEVEL
    value: info
  livenessProbe:
    httpGet:
      path: /health
      port: 8080
```

The base-and-overlays model captures environment variation, but it does not naturally capture structural similarity among related workloads. Developers therefore repeat the same fragments across multiple manifests even within a single stage.

Template systems provide another common solution. By parameterizing configuration files, they can reduce duplication across workloads and environments. However, they also make debugging harder because developers must reason about both the template logic and the generated manifests.

Figure 2 illustrates the core point: reuse spans both environment-specific variation and workload similarity. The problem is therefore not merely duplication across dev, stg, and prd. Real systems contain overlapping reuse relationships, and a practical configuration mechanism should support them directly while remaining compatible with existing tooling.

3 DESIGN PRINCIPLES BEHIND JSON++

JSON++ was designed around three principles emphasizing compatibility and simplicity.

Principle 1 — Completeness and Compatibility

Any JSON can be represented as JSON++. An important property of JSON++ is that it does not restrict the space of configurations that can be produced. The jq++ evaluation function maps JSON++ documents to ordinary JSON documents. For any valid JSON document J , there exists a JSON++ document X such that $\text{jq++}(X) = J$. In other words, the evaluation function is surjective onto JSON. This property ensures that JSON++ can generate any configuration expected by downstream systems, allowing jq++ to be introduced as a preprocessing step without reducing expressiveness.

In practice, this means that adopting JSON++ never forces downstream tools to change their expectations about the resulting configuration structure².

2. All JSON++ meta-keywords are escapable

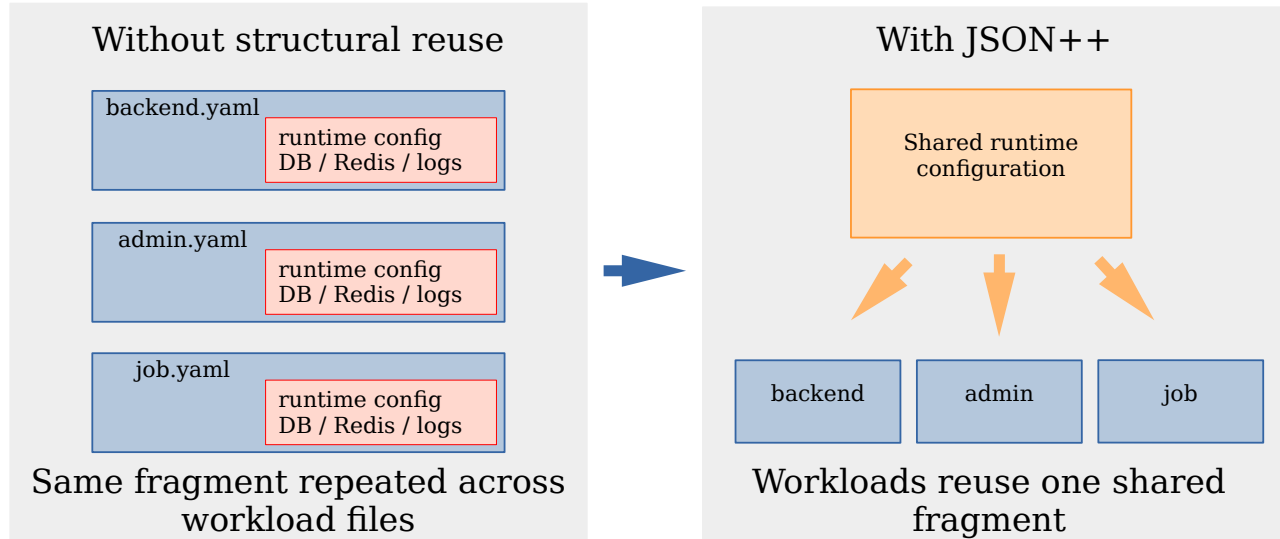


Fig. 1. Configuration duplication across related workloads. Without structural reuse, the same runtime configuration fragment is repeated in multiple workload files. With JSON++, the shared fragment can be factored out and reused declaratively.

		Environment-specific variation		
		dev	stg	prd
Workload similarity	backend	env probe db	env probe db	env probe db
	admin	env probe db	env probe db	env probe db
	job	env db	env db	env db

Fig. 2. Configuration reuse spans multiple independent dimensions. Columns represent environment-specific variation, while rows represent similarities among related workloads. In practice, duplication arises across both axes rather than along only one hierarchy.

All JSON++ documents are valid JSON. Moreover, JSON++ documents remain valid JSON, which allows existing parsers and tooling to process them without modification.

Principle 2 — Data-Model Level Extension

JSON++ operates at the level of the JSON data model rather than a specific surface syntax. Because many configuration languages such as YAML, HCL, and TOML are routinely translated into JSON during processing, the same reuse mechanisms can be applied to configurations written in those formats. In practice this means that JSON++ concepts naturally extend to JSON-compatible representations such as YAML without requiring changes to the authoring language. This also implies that the same extension concept naturally applies to other JSON-compatible formats such as YAML.

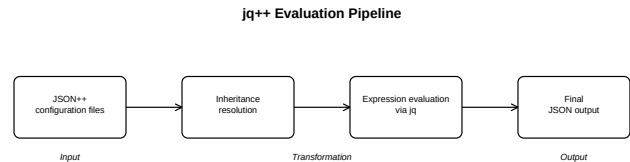


Fig. 3. jq++ evaluation pipeline. jq++ reads JSON++ configuration files, resolves inheritance, evaluates expressions using jq, and produces ordinary JSON that can be consumed by existing systems.

Principle 3 — Declarative Structural Reuse

JSON++ introduces explicit constructs for structural reuse. Configurations can extend other configurations and override selected fields. This inheritance-like mechanism allows relationships between configurations to be expressed directly in the data rather than through external template logic.

4 ARCHITECTURE OF JQ++

The jq++ tool implements the JSON++ evaluation model and produces ordinary JSON. This allows JSON++ configurations to be processed as a preprocessing step before being passed to other tools.

Figure 3 shows the evaluation pipeline: parsing JSON++ input, resolving inheritance, evaluating expressions, and emitting final JSON. Because JSON++ is syntactically compatible with JSON, the input can be generated by standard JSON emitting tools such as YAML parsers. Inheritance resolution loads referenced fragments, merges them deterministically, and applies explicit overrides. Expression

evaluation then computes derived values using jq [9]. The result is a fully expanded JSON document that downstream tools can consume directly.

This architecture separates reuse from the systems that consume the resulting configuration. Existing tools continue to operate on ordinary JSON, while jq++ acts as a dedicated preprocessing stage. To ensure predictable behavior, parent configurations are expanded before their children, multiple parents are merged in the order specified by `$extends`, and cyclic inheritance or reference relationships are rejected during evaluation.

5 IMPLEMENTATION EXPERIENCE

jq++ is implemented in Go and builds upon the widely used jq processing model. The implementation emphasizes predictable evaluation and compatibility with existing workflows. Special attention was given to deterministic resolution of inheritance relationships and to transparent error reporting, including detection of cyclic inheritance. These properties become essential when configurations are large and maintained by multiple teams.

The implementation is available as open source at <https://github.com/dakusui/jqplusplus>, together with documentation and usage examples.

During implementation, we observed that configuration reuse requires two distinct composition mechanisms. The first is inheritance, represented by the `$extends` directive, which allows a configuration to derive from one or more base definitions. The second arises when externally supplied fragments must be injected into an existing structure. In that case the goal is not derivation but augmentation, so jq++ provides a separate `$includes` directive. Distinguishing these scenarios proved important in practice, especially when platform providers distribute default configurations and individual teams supply overrides.

A second practical lesson concerned file resolution across large repositories. Many systems require imported files to be specified by relative path. While simple, that approach becomes inconvenient as configuration collections grow and are shared across projects. jq++ therefore introduces a deterministic search-path mechanism controlled by the environment variable `JF_PATH`. Referenced files are resolved by scanning directories in a fixed order, similar in spirit to `PATH`, `PYTHONPATH`, or `CLASSPATH`. Without such deterministic rules, inheritance becomes difficult to reason about in large repositories.

A remaining challenge is source-level diagnosability. Because jq++ resolves inheritance, inclusion, and expression evaluation across multiple JSON++ files before emitting a single effective document, debugging may require tracing an erroneous value in the output back to the source fragment that introduced it. Improved provenance-aware diagnostics are therefore an important direction for future tooling.

6 USING JSON++ WITH EXISTING ECOSYSTEMS

An important property of JSON++ is that it integrates naturally with ecosystems that already rely on JSON as an underlying data model. Many configuration languages, including

YAML, HCL, and TOML [10], are routinely converted into JSON during processing. Because YAML is a superset representation of JSON, the same idea can be viewed from the opposite direction as well. In practice, YAML documents interpreted under the JSON++ evaluation model effectively form a YAML++ notation. This observation highlights that JSON++ does not introduce a new configuration language but instead extends the JSON data model itself. Any language that can be mapped to JSON can benefit from the same reuse mechanisms. As a result, mechanisms defined for JSON can be applied to configurations written in these formats without modifying downstream tools.

6.1 Revisiting the Motivating Example

Section 2 illustrated a common Kubernetes pattern in which multiple workloads share similar runtime configuration. The base-and-overlays model used by tools such as Kustomize is effective for representing differences across environments. However, it does not directly address structural reuse among related workloads within the same environment. JSON++ addresses this limitation by allowing shared fragments to be factored out and inherited directly.

6.2 Example: Structuring System Configuration

Real systems contain not only duplicated structures but also operational facts such as ports and endpoint names. These values are often scattered across component definitions, making them difficult to change consistently. Using JSON++, they can be separated into reusable maps and composed into the main configuration.

Listing 3. System configuration using JSON++ abstractions

```
$local:
  ports:
    $includes: ports.yaml

  urls:
    $includes: urls.yaml

  base:
    port: eval:port_of(type())
    image: eval:image_of(type())

components:
  backend:
    $extends: [base]
    type: backend
    databaseUrl: eval:url_of("database")

  admin:
    $extends: [base]
    type: admin
    databaseUrl: eval:url_of("database")

  frontend:
    $extends: [base]
    type: frontend
    backendUrl: eval:url_of("backend")

  database:
    $extends: [base]
    type: database
```

In this representation, component blocks describe the logical topology of the system rather than concrete connection values. Operational facts such as ports and URLs

are defined once and referenced through functions such as `port_of(...)` and `url_of(...)`. Environment differences can be isolated by providing different definitions of the URL map (Figure 4). Switching environments therefore requires replacing only the environment-specific file while leaving the main configuration unchanged.

As reusable fragments and helper functions accumulate, the repository effectively becomes a small domain-specific language for describing system deployments while remaining entirely within the JSON ecosystem.

6.3 Integration into Existing Toolchains

This workflow enables incremental adoption. Engineers can continue writing configuration files in their preferred notation while introducing JSON++ only where structural reuse becomes beneficial. For example, Kubernetes manifests written in YAML can be converted to JSON, processed with jq++, and then supplied to the Kubernetes API server in standard form:

```
YAML → JSON → jq++ → JSON → Kubernetes
```

Because the output remains ordinary JSON, editors, validators, linters, and deployment systems continue to operate unchanged. This allows organizations to improve reuse without migrating to a new configuration language.

7 EXPERIENCE AND OBSERVATIONS

Our experience developing jq++ and applying JSON++ to real repositories revealed several practical observations.

7.1 Reuse Relationships Are Multi-Dimensional

In practice, configuration reuse rarely follows a single hierarchy. Instead, it arises along multiple dimensions such as environment differences, workload similarity, and organizational boundaries. Traditional overlay systems capture the first dimension well but provide limited support for the second. JSON++ proved particularly useful when these reuse relationships overlap.

7.2 Local Reuse Is Useful to Keep Files Clean

Early prototypes focused on file-level inheritance. However, practical configurations often contain repeated fragments within a single document. Supporting local reusable nodes through the `$local` construct proved essential for avoiding fragmentation into many small files.

7.3 Compatibility Strongly Influences Adoption

Users were far more willing to experiment with JSON++ when it preserved compatibility with existing tooling. Because JSON++ documents remain valid JSON and jq++ emits ordinary JSON, editors, validators, and deployment systems required no modification. This allowed incremental adoption within existing pipelines.

7.4 Preprocessing Pipelines Fit Existing Workflows

The jq++ preprocessing model integrates naturally into UNIX-style pipelines. Configurations can be generated, transformed, and validated through a sequence of composable tools. This architecture proved easier to integrate into existing CI/CD workflows than approaches that require dedicated configuration runtimes.

7.5 Configuration Repositories Tend to Evolve into DSLs

In several projects, repositories using reusable fragments and helper functions gradually evolved into small domain-specific languages. Over time, configuration files described the logical structure of the system more than concrete values. Developers increasingly interacted with named abstractions such as reusable definitions and helper functions, rather than repeating literal values throughout the repository.

8 LESSONS LEARNED

The development of JSON++ and jq++ provided several lessons about how configuration abstractions should be introduced into existing ecosystems.

8.1 Reuse Must Reflect Real Configuration Structure

Real repositories exhibit overlapping reuse relationships such as environment variation, service similarity, and organizational boundaries. Tools designed around a single reuse axis cannot represent these relationships naturally. Allowing inheritance and composition within the configuration structure itself proved more flexible than relying only on external overlay systems.

8.2 Different Composition Scenarios Require Different Mechanisms

One scenario derives a configuration from a base definition, while another augments an existing structure with externally supplied fragments. Modeling both situations with a single mechanism led to ambiguous semantics. Introducing separate directives for derivation (`$extends`) and augmentation (`$includes`) simplified the model and clarified merge behavior.

8.3 Expression Evaluation Enables Small Domain-Specific Abstractions

Allowing configuration values to be computed using jq expressions enabled users to encode small domain-specific rules directly in configuration files. Typical examples included deriving URLs, constructing environment-dependent identifiers, and generating repeated structures. As a result, configuration files stayed close to their final structure while still supporting limited computation.

This effect becomes clearer when reusable fragments are factored by concern. For example, separate files can define port assignments, logical endpoint names, and system topology, each of which begins to resemble a small declarative vocabulary of its own (Figure 4).

ports.yaml

```
backend: 8080
admin: 8081
job: 8082
database: 5432
redis: 6379
```

Port-assignment DSL

urls.yaml

```
backend: backend
admin: admin
job: job
database: postgres
redis: redis
```

Endpoint-naming DSL

system.yaml

```
$local:
  ports:
    $includes: [ports.yaml]
  urls:
    $includes: [urls.yaml]
  baseService:
    port: 'eval:port_of(type())'
    databaseUrl: 'eval:url_of("database")'
    cacheUrl: 'eval:url_of("redis")'

  services:
    backend:
      $extends: [baseService]
      type: backend
    admin:
      $extends: [baseService]
      type: admin
    job:
      $extends: [baseService]
      type: job
```

Topology DSL

Fig. 4. Factored configuration fragments can evolve into small domain-specific declarative vocabularies. Here, `ports.yaml` defines port assignments, `urls.yaml` defines logical service endpoint names, and `system.yaml` describes system topology by referring to those abstractions. Rather than merely removing duplication, JSON++ allows repositories to separate concerns into compact, JSON-compatible fragments that resemble lightweight DSLs.

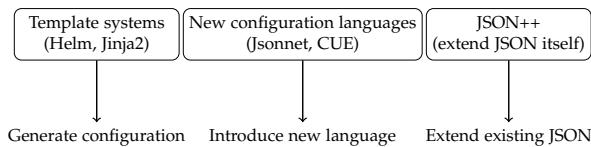


Fig. 5. Three common approaches to configuration abstraction. Template systems generate configuration through embedded template languages, while systems such as Jsonnet and CUE introduce new configuration languages. JSON++ takes a different approach by extending JSON itself with lightweight declarative reuse mechanisms.

9 RELATED WORK

Several approaches have been proposed to reduce duplication and improve modularity in configuration systems.

Figure 5 summarizes the design space of existing approaches.

One class of solutions introduces dedicated configuration or data-templating languages that generate structured configuration data. Jsonnet [4] provides functions, imports, and inheritance-like mechanisms for constructing configuration objects. CUE [5] combines configuration with constraint validation and schema-like typing. These systems are powerful, but they require users to adopt languages and toolchains distinct from JSON.

Template-based systems follow a similar philosophy. Tools such as Helm templates [11], Jinja2 [12], Handlebars, and ERB generate configuration files by embedding a template language into the source representation. Although flexible, they introduce another language layer and require developers to reason about both the template logic and the generated configuration. Because the source is no longer plain JSON, JSON-oriented tooling cannot be applied directly.

Another widely used strategy is overlay-based composition. Kustomize applies patches or overlays to a base

configuration, typically to represent differences across environments. This model is effective for environment variation but provides limited support for expressing structural reuse among related objects within the same hierarchy.

JSON++ differs from these approaches by preserving JSON as the authoring format and introducing declarative reuse directly within the configuration structure. Rather than defining a new language or embedding a template system, JSON++ adds lightweight annotations that remain compatible with standard JSON syntax. The `jq++` tool evaluates these annotations through a preprocessing stage and emits ordinary JSON that existing tools can consume directly.

Key Takeaways

- Real configuration repositories contain overlapping reuse relationships, not just environment-specific differences.
- A small extension to JSON can support structural reuse without introducing a new authoring language.
- By processing and emitting ordinary JSON, `jq++` fits naturally into existing configuration and deployment pipelines.

10 CONCLUSION

Configuration reuse remains a persistent challenge in modern software systems. As systems grow in size and complexity [13], configuration repositories accumulate duplicated fragments that are difficult to maintain and evolve.

This article introduced JSON++, a lightweight mechanism for expressing structural reuse directly within JSON configurations. JSON++ adds two minimal abstractions—inheritance and expression evaluation—while preserving compatibility with the JSON data model. We also presented

jq++, a tool that evaluates JSON++ configurations and produces ordinary JSON. Because both input and output remain standard JSON, existing tools such as editors, validators, and deployment systems can continue to operate unchanged.

Our experience suggests that effective configuration reuse requires mechanisms that reflect the true structure of configuration repositories. In practice, reuse relationships arise across multiple dimensions, including environments, services, and organizational boundaries. By embedding reuse directly in the configuration structure, JSON++ provides a flexible and declarative way to represent these relationships.

More broadly, JSON++ demonstrates that expressive configuration abstractions can be introduced without inventing new configuration languages. Small, compatible extensions to widely adopted formats can provide significant practical benefits while minimizing disruption to existing workflows. Future work will focus on source-aware diagnostics, provenance tracking, and broader tooling support, as well as adjacent domains such as testing and game content definition.

ACKNOWLEDGMENT

The author acknowledges the assistance of OpenAI’s ChatGPT for language editing and structural refinement of the manuscript. All technical ideas, design decisions, and conclusions are the author’s own.

REFERENCES

- [1] T. Bray, “The JavaScript Object Notation (JSON) Data Interchange Format,” IETF RFC 8259, 2017. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8259>
- [2] O. Ben-Kiki, C. Evans, and I. dot Net, “YAML Ain’t Markup Language (YAML) Version 1.2,” Available at <https://yaml.org/spec/1.2/spec.html>, 2009.
- [3] HashiCorp, “HashiCorp Configuration Language (HCL),” Available at <https://github.com/hashicorp/hcl>, 2014.
- [4] Jsonnet Authors, “Jsonnet specification,” <https://jsonnet.org/ref/spec.html>, accessed 2025-09-11.
- [5] The CUE Authors, “The cue language specification,” <https://cuelang.org/docs/reference/spec/>, 2025, accessed 2025-09-11.
- [6] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, omega, and kubernetes,” in *Proceedings of the Tenth European Conference on Computer Systems*, ser. EuroSys ’15. ACM, 2015.
- [7] The Kubernetes Authors, “Kubernetes: Production-grade container orchestration,” <https://kubernetes.io/>, 2015, accessed: 2026-03-19.
- [8] Kubernetes SIG CLI, “Kustomize: Kubernetes native configuration customization,” <https://github.com/kubernetes-sigs/kustomize>, 2018, accessed: 2026-03-19.
- [9] S. Dolan *et al.*, “jq: A lightweight and flexible command-line JSON processor,” Available at <https://stedolan.github.io/jq/>, 2012.
- [10] T. Preston-Werner, “TOML: Tom’s Obvious, Minimal Language,” Available at <https://toml.io/en/>, 2013.
- [11] The Helm Authors, “Helm: The kubernetes package manager,” <https://helm.sh/>, 2015, accessed: 2025-09-10.
- [12] A. Ronacher, “Jinja2: A modern and designer-friendly templating language for python,” <https://palletsprojects.com/p/jinja/>, 2008, accessed: 2025-09-06.
- [13] T. C. CTO, “Working with terraform: 10 months in,” *The Consulting CTO blog*, 10 2017, describes how writing over 40,000 lines of HCL exposed issues of code duplication, low composability, and the difficulty of maintaining Terraform configurations. [Online]. Available: <https://theconsultingcto.com/posts/working-with-terraform-10-months-in/>